

Day 3 SQL scorecard with 3-day trend and corrections

DAY 3 RESULTS



Dip today — and that's expected

Day 3 jumped into genuinely advanced territory (recursive CTEs, MERGE, SCD2) and two questions were skipped entirely. The score drop reflects difficulty, not regression — Q1 and Q8 were excellent.

Full marks	Partial	Failed / skip	vs Day 2
2	6	2	-1.6

3-DAY TREND

Day 1	6.0 / 10
Day 2	6.9 / 10
Day 3	5.3 / 10

QUESTION-BY-QUESTION BREAKDOWN

Q1 — String functions

★★★★★ 10/10

Perfect. Nested SPLIT_PART used correctly for both the '|' delimiter and the ':' delimiter. This is exactly the redemption I wanted to see after Day 2's skip. Note: in production you'd also CAST the timestamp string to TIMESTAMP type, but the extraction logic is fully correct.

Q2 — Window frame clauses

★★★☆☆ 4/10

Right concept — AVG() as a window function is correct. Syntax broken: "ROW BETWEEN sale_date, 2 PRECEDING" isn't valid — it's ROWS BETWEEN 2 PRECEDING AND CURRENT ROW, no column name needed since it operates on physical rows, not date values. Also missing WHERE for 2023.

```
SELECT sale_date,
ROUND(AVG(total_amount) OVER (
ORDER BY sale_date
ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
), 2) AS moving_avg
FROM daily_sales
WHERE EXTRACT(YEAR FROM sale_date) = 2023
ORDER BY sale_date;
```

Q3 — JOIN on aggregates

★★★☆☆ 3/10

Two issues: (1) The question explicitly asked to avoid window functions — you used MAX() OVER(), which is exactly the technique to avoid (good for practice, but doesn't answer what was asked). (2) WHERE department=temp.department compares a table to itself and the subquery's MAX column has no alias, so this wouldn't even run.

```

SELECT e.department, e.name, e.salary
FROM employees e
JOIN (
  SELECT department, MAX(salary) AS max_salary
  FROM employees
  GROUP BY department
) m ON e.department = m.department AND e.salary = m.max_salary;

```

Q4 — Recursive CTE

☆☆☆☆☆ 0/10

Skipped — this was flagged as the trickiest concept today, and it's worth sitting with it rather than skipping. Study this pattern closely, it appears constantly in DE interviews for org charts, BOMs, and category trees:

```

WITH RECURSIVE hierarchy AS (
  -- Anchor: top-level managers
  SELECT id, name, manager_id, 1 AS level
  FROM employees
  WHERE manager_id IS NULL

  UNION ALL

  -- Recursive: join next level down
  SELECT e.id, e.name, e.manager_id, h.level + 1
  FROM employees e
  JOIN hierarchy h ON e.manager_id = h.id
)
SELECT h.id, h.name, m.name AS manager_name, h.level
FROM hierarchy h
LEFT JOIN employees m ON h.manager_id = m.id
ORDER BY h.level, h.id;

```

Q5 — ETL transformation

☆☆☆☆☆ 3/10

Right transformations identified — TRIM, UPPER, NULL handling, age filter are all the correct ideas. Syntax is broken: (1) **INSERT INTO ... VALUES(SELECT ...)** is invalid — INSERT INTO ... SELECT doesn't use VALUES or parentheses around the SELECT. (2) **REPLACE(email, NULL, ...)** is wrong — REPLACE does string substitution, not NULL handling; use COALESCE. (3) Typo: "country_coode".

```

INSERT INTO clean_users (id, name, email, country_code, age)
SELECT id,
  TRIM(name),
  COALESCE(email, 'unknown@domain.com'),
  UPPER(country_code),
  age
FROM staging
WHERE age IS NOT NULL AND age >= 18;

```

Q6 — Percentile & NTILE

☆☆☆☆☆ 5/10

Good use of NTILE(4) in the right CTE structure. Bugs: (1) JOIN orders o ON t.customer_id, o.customer_id is invalid join syntax (needs =) and re-joining orders duplicates rows, breaking the per-customer total. (2) Only returned quartile 1 and 4 instead of all 4. (3) Missing the MIN/MAX per quartile requirement entirely.

```
WITH total_spend AS (
  SELECT customer_id, SUM(amount) AS spend_amount
  FROM orders GROUP BY customer_id
),
quartiled AS (
  SELECT customer_id, spend_amount,
  NTILE(4) OVER (ORDER BY spend_amount) AS qrt
  FROM total_spend
)
SELECT qrt,
  MIN(spend_amount) AS min_spend,
  MAX(spend_amount) AS max_spend
FROM quartiled
GROUP BY qrt
ORDER BY qrt;
```

Q7 — SCD Type 2 logic

★★★★☆ 8/10

Very strong — the self-join with effective_date inequality and email inequality is exactly the right SCD2 pattern. Minor fix: single quotes around column aliases ('old_email') should be double quotes or backticks depending on dialect — single quotes make it a string literal, not an alias, in most engines.

```
SELECT ch.customer_id,
  ch.email AS old_email,
  c.email AS new_email,
  c.effective_date
FROM customer_history ch
JOIN customer_history c
  ON ch.customer_id = c.customer_id
  AND ch.effective_date < c.effective_date
  AND ch.email <> c.email
ORDER BY ch.effective_date;
```

Q8 — Set operations

★★★★★ 10/10

Flawless. INTERSECT, EXCEPT, and UNION used exactly right for the three set logic questions. Clean and idiomatic.

Q9 — Aggregation with FILTER

★★★☆☆ 4/10

Creative idea using WITH ROLLUP for the grand total — shows you know an alternate technique. But it doesn't answer the question: the requirement was all four values (total, 2023 count, 2024 count, pct) in ONE row — your approach needs 3 separate queries and never computes the percentage. The FILTER/CASE approach is more direct here.

```
SELECT
  COUNT(*) AS total_orders,
  COUNT(*) FILTER (WHERE EXTRACT(YEAR FROM order_date) = 2023) AS orders_2023,
  COUNT(*) FILTER (WHERE EXTRACT(YEAR FROM order_date) = 2024) AS orders_2024,
```

```
ROUND(100.0 * COUNT(*) FILTER (WHERE EXTRACT(YEAR FROM order_date) = 2024) / COUNT(*), 2) AS pct_2024
FROM orders;
```

Q10 — Pipeline idempotency (MERGE)

☆☆☆☆☆ 0/10

Skipped. This is one of the most asked DE interview questions — every modern warehouse (Snowflake, BigQuery, Databricks) uses MERGE for incremental loads. Study this pattern carefully:

```
MERGE INTO target_orders t
USING source_orders s
ON t.order_id = s.order_id
WHEN MATCHED THEN
  UPDATE SET t.amount = s.amount,
            t.updated_at = CURRENT_TIMESTAMP
WHEN NOT MATCHED THEN
  INSERT (order_id, customer_id, amount, order_date, updated_at)
  VALUES (s.order_id, s.customer_id, s.amount, s.order_date, CURRENT_TIMESTAMP);
```

What stood out today

Q1, Q7, and Q8 show you can absolutely write correct, production-quality SQL when the pattern is in your hands — nested string parsing, SCD2 self-joins, and set operations were clean. The dip in score is concentrated in syntax slips (INSERT...VALUES(SELECT), invalid JOIN ON) and two skips, not in conceptual misunderstanding.

Focus for Day 4 — non-negotiable

1. Recursive CTEs — don't skip again. Write the anchor + recursive structure from memory 3 times tomorrow before the test.
2. MERGE/UPSERT syntax — memorize the WHEN MATCHED / WHEN NOT MATCHED skeleton cold.
3. INSERT INTO ... SELECT — never wrap SELECT in VALUES().
4. JOIN conditions always use =, never a comma.
5. Read "return ALL X" carefully — Q6 and Q9 both lost points for returning a subset instead of the full requested output.

Start Day 4 ↗